

Scikit-learn (sklearn) – Full Concept Guide

1. What is Scikit-learn?

Scikit-learn is one of the most popular Python libraries for **Machine Learning (ML)**. It provides simple and efficient tools for:

- Classification
- Regression
- Clustering
- Dimensionality Reduction
- Model Selection
- Data Preprocessing

It is built on:

- **NumPy**
 - **SciPy**
 - **Matplotlib**
-

2. Installation

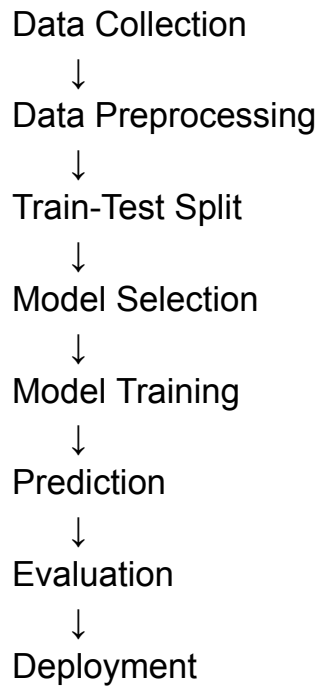
```
pip install scikit-learn
```

Check version:

```
import sklearn
```

```
print(sklearn.__version__)
```

3. Machine Learning Workflow



4. Datasets in Scikit-learn

Built-in Datasets

```
from sklearn.datasets import load_iris
```

```
data = load_iris()
```

```
print(data.data)  
print(data.target)
```

Popular datasets:

Dataset	Purpose
Iris	Classification

Wine	Classificati on
Breast Cancer	Classificati on
Diabetes	Regression
Digits	Classificati on

5. Train-Test Split

Used to divide data into training and testing sets.

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.2,  
    random_state=42  
)
```

Parameters

- test_size
 - train_size
 - random_state
 - shuffle
-

6. Data Preprocessing

Standardization

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

Formula:

$$z = \frac{x - \mu}{\sigma}$$

Normalization

```
from sklearn.preprocessing import MinMaxScaler
```

```
scaler = MinMaxScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

Range: 0 to 1

Label Encoding

```
from sklearn.preprocessing import LabelEncoder
```

```
le = LabelEncoder()
```

```
y = le.fit_transform(y)
```

One Hot Encoding

```
from sklearn.preprocessing import OneHotEncoder
```

```
encoder = OneHotEncoder()
```

```
result = encoder.fit_transform(data)
```

7. Classification Algorithms

Classification predicts categories.

Example:

- Spam/Not Spam
 - Pass/Fail
 - Disease/No Disease
-

A. Logistic Regression

```
from sklearn.linear_model import LogisticRegression
```

```
model = LogisticRegression()
```

```
model.fit(X_train,y_train)
```

```
prediction = model.predict(X_test)
```

Advantages:

- Fast
 - Simple
 - Good baseline model
-

B. K Nearest Neighbors (KNN)

```
from sklearn.neighbors import KNeighborsClassifier
```

```
model = KNeighborsClassifier(n_neighbors=3)
```

```
model.fit(X_train,y_train)
```

Works using nearest neighbors.

C. Decision Tree

```
from sklearn.tree import DecisionTreeClassifier
```

```
model = DecisionTreeClassifier()
```

```
model.fit(X_train,y_train)
```

Creates tree-like decisions.

D. Random Forest

```
from sklearn.ensemble import RandomForestClassifier
```

```
model = RandomForestClassifier()
```

```
model.fit(X_train,y_train)
```

Collection of multiple decision trees.

E. Support Vector Machine (SVM)

```
from sklearn.svm import SVC
```

```
model = SVC()
```

```
model.fit(X_train,y_train)
```

Finds best separating boundary.

F. Naive Bayes

```
from sklearn.naive_bayes import GaussianNB
```

```
model = GaussianNB()
```

```
model.fit(X_train,y_train)
```

Based on probability.

8. Regression Algorithms

Regression predicts continuous values.

Examples:

- House Price
 - Salary Prediction
 - Temperature Forecast
-

Linear Regression

```
from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression()
```

```
model.fit(X_train,y_train)
```

Prediction:

```
y_pred = model.predict(X_test)
```

Polynomial Regression

```
from sklearn.preprocessing import PolynomialFeatures
```

Used for curved relationships.

Decision Tree Regressor

```
from sklearn.tree import DecisionTreeRegressor
```

Random Forest Regressor

```
from sklearn.ensemble import RandomForestRegressor
```

9. Clustering

Unsupervised Learning

No target variable.

K-Means Clustering

```
from sklearn.cluster import KMeans
```

```
model = KMeans(n_clusters=3)
```

```
model.fit(X)
```


Predict cluster:

```
model.predict(X)
```

Applications:

- Customer Segmentation
 - Market Analysis
-

Hierarchical Clustering

```
from sklearn.cluster import AgglomerativeClustering
```

```
model = AgglomerativeClustering(n_clusters=3)
```

10. Dimensionality Reduction

Reduce number of features.

PCA

```
from sklearn.decomposition import PCA
```

```
pca = PCA(n_components=2)
```

```
X_pca = pca.fit_transform(X)
```

Benefits:

- Faster training
- Less memory
- Better visualization

11. Model Evaluation

Accuracy

```
from sklearn.metrics import accuracy_score
```

```
accuracy_score(y_test,y_pred)
```

Confusion Matrix

```
from sklearn.metrics import confusion_matrix
```

```
cm = confusion_matrix(y_test,y_pred)
```

Classification Report

```
from sklearn.metrics import classification_report
```

```
print(classification_report(y_test,y_pred))
```

Contains:

- Precision
 - Recall
 - F1 Score
-

Mean Absolute Error

```
from sklearn.metrics import mean_absolute_error
```

Mean Squared Error

```
from sklearn.metrics import mean_squared_error
```

R² Score

```
from sklearn.metrics import r2_score
```

12. Cross Validation

More reliable model evaluation.

```
from sklearn.model_selection import cross_val_score
```

```
scores = cross_val_score(  
    model,  
    X,  
    y,  
    cv=5  
)
```

```
print(scores)
```

13. Hyperparameter Tuning

Grid Search

```
from sklearn.model_selection import GridSearchCV
```

```
params = {  
    'n_neighbors':[3,5,7,9]  
}
```

```
grid = GridSearchCV(  
    KNeighborsClassifier(),  
    params,  
    cv=5  
)
```

```
grid.fit(X,y)
```

```
print(grid.best_params_)
```

Random Search

```
from sklearn.model_selection import RandomizedSearchCV
```

Faster than Grid Search.

14. Pipelines

Automates preprocessing + model training.

```
from sklearn.pipeline import Pipeline  
from sklearn.preprocessing import StandardScaler  
from sklearn.svm import SVC
```

```
pipe = Pipeline([  
    ('scaler',StandardScaler()),  
    ('svm',SVC())  
])
```

```
pipe.fit(X_train,y_train)
```

15. Feature Selection

Choose important features.

```
from sklearn.feature_selection import SelectKBest
```

Benefits:

- Faster training
 - Better accuracy
 - Less overfitting
-

16. Saving Model

```
import joblib
```

```
joblib.dump(model,'model.pkl')
```

Load:

```
model = joblib.load('model.pkl')
```

17. Complete Example

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
```

```
data = load_iris()
```

```
X = data.data
```

```
y = data.target
```

```
X_train,X_test,y_train,y_test = train_test_split(
    X,
    y,
```

```
test_size=0.2,  
random_state=42  
)  
  
model = RandomForestClassifier()  
  
model.fit(X_train,y_train)  
  
y_pred = model.predict(X_test)  
  
print("Accuracy:",  
      accuracy_score(y_test,y_pred))
```

Important Interview Questions

1. What is Scikit-learn?
 2. Difference between Classification and Regression?
 3. What is Train-Test Split?
 4. What is Overfitting?
 5. What is Underfitting?
 6. What is Cross Validation?
 7. What is GridSearchCV?
 8. Difference between Random Forest and Decision Tree?
 9. What is PCA?
 10. What is Pipeline?
 11. What is Feature Selection?
 12. What is StandardScaler?
 13. What is Label Encoding?
 14. What is Confusion Matrix?
 15. What is Precision, Recall, F1 Score?
-

Scikit-learn Modules Cheat Sheet

Module	Purpose
--------	---------

sklearn.datasets	Datasets
sklearn.preprocessing	Data Cleaning
sklearn.model_selection	Train/Test Split
sklearn.linear_model	Linear Models
sklearn.neighbors	KNN
sklearn.tree	Decision Trees
sklearn.ensemble	Random Forest
sklearn.svm	SVM
sklearn.cluster	Clustering
sklearn.decomposition	PCA
sklearn.metrics	Evaluation
sklearn.pipeline	Pipeline
sklearn.feature_selection	Feature Selection

Learning order: NumPy → Pandas → Matplotlib → Scikit-learn → Machine Learning Projects. This sequence makes Scikit-learn much easier to understand and use effectively.

Scikit-learn (sklearn) Library – Theory and Practical Basics

What is Scikit-learn?

Scikit-learn (sklearn) is an open-source Python library used for **Machine Learning (ML)**. It provides simple and efficient tools for data analysis, prediction, classification, clustering, and model evaluation.

Why Use Scikit-learn?

- Easy to learn and use
 - Built on NumPy and SciPy
 - Supports various ML algorithms
 - Provides preprocessing tools
 - Includes model evaluation metrics
 - Widely used in industry and research
-

Prerequisites

Before learning sklearn, you should know:

1. Python Basics
 2. NumPy
 3. Pandas
 4. Basic Statistics
 5. Machine Learning Concepts
-

Installation

```
pip install scikit-learn
```

Check Version:

```
import sklearn
```

```
print(sklearn.__version__)
```

Machine Learning Types in Scikit-learn

1. Supervised Learning

The model learns from labeled data.

Examples:

- House Price Prediction
- Student Marks Prediction
- Spam Email Detection

Algorithms:

- Linear Regression
 - Logistic Regression
 - Decision Tree
 - Random Forest
 - KNN
 - SVM
-

2. Unsupervised Learning

The model learns from unlabeled data.

Examples:

- Customer Segmentation
- Pattern Recognition

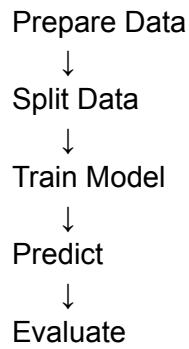
Algorithms:

- K-Means Clustering
 - Hierarchical Clustering
-

Basic Workflow of Scikit-learn

Collect Data





Important Modules

Module	Purpose
<code>sklearn.model_selection</code>	Data splitting
<code>sklearn.linear_model</code>	Regression models
<code>sklearn.tree</code>	Decision Tree
<code>sklearn.ensemble</code>	Random Forest
<code>sklearn.neighbors</code>	KNN
<code>sklearn.svm</code>	Support Vector Machine
<code>sklearn.cluster</code>	Clustering
<code>sklearn.metrics</code>	Evaluation
<code>sklearn.preprocessing</code>	Data preprocessing

Practical 1: Train-Test Split

Theory

Data is divided into:

- Training Data (80%)
- Testing Data (20%)

Code

```
from sklearn.model_selection import train_test_split

X = [[1],[2],[3],[4],[5]]
y = [10,20,30,40,50]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(X_train)
print(X_test)
```

Practical 2: Linear Regression

Theory

Used when the output is continuous.

Example:

- House Price Prediction
- Salary Prediction

Code

```
from sklearn.linear_model import LinearRegression

X = [[1],[2],[3],[4],[5]]
y = [10,20,30,40,50]

model = LinearRegression()

model.fit(X, y)

prediction = model.predict([[6]])

print(prediction)
```

Output:

```
[60.]
```

Practical 3: Logistic Regression

Theory

Used for Classification Problems.

Examples:

- Pass/Fail
- Spam/Not Spam
- Yes/No

Code

```
from sklearn.linear_model import LogisticRegression
```

```
X = [[1],[2],[3],[4]]
```

```
y = [0,0,1,1]
```

```
model = LogisticRegression()
```

```
model.fit(X, y)
```

```
print(model.predict([[3]]))
```

Practical 4: Decision Tree

Theory

Creates decisions using tree-like structures.

Example:

Age > 18 ?

Yes → Adult

No → Child

Code

```
from sklearn.tree import DecisionTreeClassifier
```

```
X = [[1],[2],[3],[4]]
```

```
y = [0,0,1,1]
```

```
model = DecisionTreeClassifier()
```

```
model.fit(X, y)
```

```
print(model.predict([[3]]))
```

Practical 5: K-Nearest Neighbors (KNN)

Theory

Predicts based on nearest neighbors.

Code

```
from sklearn.neighbors import KNeighborsClassifier
```

```
X = [[1],[2],[3],[4]]
```

```
y = [0,0,1,1]
```

```
model = KNeighborsClassifier(n_neighbors=3)
```

```
model.fit(X, y)
```

```
print(model.predict([[3]]))
```

Practical 6: Data Scaling

Theory

Scaling makes data values comparable.

Example:

Age = 25

Salary = 50000

Different ranges can affect models.

Code

```
from sklearn.preprocessing import StandardScaler
```

```
data = [[10],[20],[30],[40]]

scaler = StandardScaler()

scaled_data = scaler.fit_transform(data)

print(scaled_data)
```

Practical 7: Accuracy Score

Theory

Measures model performance.

Formula:

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Predictions}}$$

Code

```
from sklearn.metrics import accuracy_score

y_true = [1,0,1,1]
y_pred = [1,0,1,0]

print(accuracy_score(y_true, y_pred))
```

Output:

0.75

Practical 8: Random Forest

Theory

Combination of multiple Decision Trees.

Advantages:

- High Accuracy

- Less Overfitting
- Robust Model

Code

```
from sklearn.ensemble import RandomForestClassifier
```

```
X = [[1],[2],[3],[4]]
```

```
y = [0,0,1,1]
```

```
model = RandomForestClassifier()
```

```
model.fit(X, y)
```

```
print(model.predict([[3]]))
```

Frequently Asked Interview Questions

1. What is Scikit-learn?

A Python machine learning library used for building and evaluating ML models.

2. Is sklearn used for Deep Learning?

No. Deep Learning is mainly done using TensorFlow and PyTorch.

3. What is train_test_split?

A function used to divide data into training and testing sets.

4. What is fit()?

Used to train a model.

5. What is predict()?

Used to make predictions on new data.

6. What is StandardScaler?

A preprocessing tool that standardizes data before training.

Summary

Must-Know Basic Topics of sklearn

1. Installation
2. train_test_split
3. Linear Regression
4. Logistic Regression
5. Decision Tree
6. KNN
7. Random Forest
8. StandardScaler
9. Accuracy Score
10. fit(), predict(), score()

These topics form the foundation of almost every Machine Learning project built with Scikit-learn.